

Livello Rete – Risposte Dettagliate

Sezione 5.1 – Problemi Architettureali dello Stato Rete

1. Qual è lo scopo principale del livello di rete?

Lo scopo principale del livello di rete è interconnettere tra loro i diversi collegamenti punto-punto, in modo da poter mettere in comunicazione anche dispositivi geograficamente lontani. Le informazioni che viaggiano nei pacchetti sono trasmesse dal mittente alla destinazione attraversando tutti i nodi della rete che si trovano sul tragitto congiungente questi due. Il livello di rete si occupa di instradare i pacchetti dalla sorgente alla destinazione attraverso una serie di nodi intermedi (router), rendendo possibile la comunicazione anche tra reti diverse e geograficamente distanti.

2. Cosa significa l'architettura "store and forward" nel livello di rete?

L'architettura "store and forward" (immagazzina e inoltra) significa che i pacchetti sono interamente ricevuti da ogni nodo prima di essere ritrasmessi. In pratica, quando un pacchetto arriva a un router, questo lo riceve completamente, lo memorizza temporaneamente, ne verifica l'integrità (attraverso il checksum dell'header), consulta la propria tabella di routing per determinare la prossima destinazione, e solo dopo aver completato tutte queste operazioni lo ritrasmette sul collegamento in uscita. Questo approccio garantisce che i pacchetti vengano inoltrati solo dopo essere stati ricevuti correttamente, ma introduce anche una latenza aggiuntiva dovuta al tempo di immagazzinamento ed elaborazione in ciascun nodo.

3. Quali sono le due scuole di pensiero contrastanti riguardo al servizio che la rete dovrebbe fornire al livello di trasporto?

Le due scuole di pensiero contrastanti sono:

1. **Servizio orientato alla connessione:** sostenuta da coloro che fanno riferimento all'esperienza delle reti telefoniche, vorrebbero che la rete fornisse un servizio affidabile, in cui la QoS (Quality of Service) è un fattore fondamentale. Secondo questa visione, una rete senza connessioni non potrebbe fornire un servizio di qualità, soprattutto per applicazioni Real Time (come voce e video in tempo reale).
2. **Servizio non orientato alla connessione (reti a datagram):** sostenuta da coloro che hanno sviluppato Internet nel tempo. Per loro, l'unica preoccupazione del livello di rete dovrebbe essere quella di accettare e inoltrare i pacchetti, ognuno completo di indirizzo destinazione e mittente. Secondo questa visione, operazioni come l'ordinamento e il controllo di flusso non dovrebbero appartenere a questo livello in quanto già implementate a livello superiore (trasporto).

4. Che differenza c'è tra una rete orientata alla connessione e una non orientata alla connessione?

La differenza fondamentale è la seguente:

- **Rete orientata alla connessione:** prima di iniziare la trasmissione dei dati, viene stabilito un circuito virtuale o un percorso dedicato tra mittente e destinatario. Durante questa fase di setup, i nodi della rete mantengono informazioni sullo stato della connessione. Questo permette di garantire qualità del

servizio (QoS), affidabilità e risorse dedicate. È il modello tipico delle reti telefoniche tradizionali e di alcuni protocolli come ATM.

- **Rete non orientata alla connessione (reti a datagram)** : non esiste una fase di setup preliminare. Ogni pacchetto (datagramma) viene inoltrato in maniera indipendente dagli altri, e i router non mantengono alcuna informazione sullo stato delle connessioni. Ogni datagramma contiene già al suo interno gli indirizzi completi di mittente e destinatario. Questo rende difficile garantire QoS e controllo della congestione, ma rende la rete più robusta a guasti e più semplice da gestire. È il modello tipico di Internet (IP).

5. Cosa si intende per "rete a datagram"? Quali informazioni contiene un datagramma?

Una rete a datagram è una rete non orientata alla connessione, in cui l'unità di informazione è proprio il datagramma che si muove tra i nodi della rete. Il datagramma contiene sempre gli indirizzi del destinatario e del mittente, che possono essere di 32 bit (IPv4) o 128 bit (IPv6). In queste reti, poiché ogni datagramma è inoltrato in maniera indipendente dagli altri, i nodi che si trovano sul percorso attraversato (router) non mantengono alcuna informazione sullo stato delle connessioni. Ogni datagramma contiene anche altre informazioni come il numero di sequenza (per la ricostruzione dell'ordine), il TTL (Time To Live), e un checksum per il controllo degli errori dell'header.

6. Perché nelle reti a datagram è difficile garantire Qualità del Servizio (QoS) e controllo della congestione?

Nelle reti a datagram è difficile garantire QoS e controllo della congestione perché i router non mantengono alcuna informazione sullo stato delle connessioni. Poiché ogni pacchetto viene inoltrato indipendentemente dagli altri, il router non sa a quale flusso appartiene un pacchetto né quali risorse sono state già allocate per quel flusso. Non c'è quindi modo di riservare banda, garantire una latenza massima, o applicare politiche differenziate ai diversi flussi. Inoltre, senza informazioni sullo stato delle connessioni, è complesso rilevare situazioni di congestione riferite a specifici flussi e applicare meccanismi di controllo come la limitazione della velocità di trasmissione o la priorità selettiva dei pacchetti.

7. Quale vantaggio offre una rete a datagram in caso di malfunzionamento di un router?

Il vantaggio principale di una rete a datagram in caso di malfunzionamento di un router è che non ci sarebbero grossi problemi sulla connessione in atto, se non la perdita dei datagrammi presenti nel router al momento del malfunzionamento. Poiché non esiste una connessione prestabilita che dipende da un percorso fisso, i pacchetti successivi potranno semplicemente seguire percorsi alternativi (grazie agli algoritmi di routing dinamico) per raggiungere la destinazione. Non è necessario ristabilire alcuna connessione né notificare l'accaduto ai nodi sorgente o destinatario. Questo rende la rete a datagram molto più robusta e resiliente ai guasti rispetto a una rete orientata alla connessione, dove il guasto di un nodo intermedio interrompe l'intera connessione.

Sezione 5.2 – Protocollo IPv4 e Indirizzamento

8. Perché il protocollo IP è definito "connectionless" e "inaffidabile"? Elenca almeno quattro caratteristiche.

IP è definito **connectionless** (senza connessione) perché tratta ogni pacchetto in maniera indipendente, senza stabilire un percorso prestabilito né mantenere informazioni di stato per la comunicazione tra mittente e destinatario.

IP è definito **inaffidabile** (unreliable) perché non garantisce la consegna dei pacchetti, non garantisce la correttezza dell'informazione trasportata, e non richiede acknowledgment da alcun host.

Le quattro caratteristiche che lo definiscono sono:

1. **Tratta ogni pacchetto in maniera indipendente** – non c'è relazione di stato tra pacchetti successivi dello stesso flusso.
2. **Non garantisce la consegna** – i pacchetti possono essere scartati (ad esempio per congestione o errori) senza che IP faccia nulla per recuperarli.
3. **Non garantisce la correttezza dell'informazione trasportata** – il checksum protegge solo l'header, non i dati (payload). Eventuali errori nei dati non vengono corretti a livello IP.
4. **Non richiede acknowledgment da alcun host** – né dal destinatario né da nodi intermedi. IP non prevede meccanismi di conferma di ricezione; se un pacchetto arriva o viene perso, IP non lo sa e non lo comunica.

9. In IPv4, i pacchetti possono seguire percorsi diversi e arrivare in ordine diverso da quello di invio. A quale livello superiore spetta riordinarli?

Poiché IP è un protocollo non orientato alla connessione e un messaggio (come un file o un'email) è suddiviso in un certo numero di pacchetti, può capitare che ciascuno di essi segua sulla rete un percorso diverso. I pacchetti possono quindi arrivare in un ordine diverso da quello in cui sono stati inviati. Toccherà quindi a un **protocollo di livello superiore** (tipicamente il livello di trasporto, ad esempio TCP) rimetterli nell'ordine corretto. TCP utilizza i numeri di sequenza per riordinare i segmenti ricevuti prima di passarli all'applicazione.

10. Qual è la dimensione minima e massima dell'header di un pacchetto IPv4 (in byte)?

L'intestazione (header) del pacchetto IPv4 ha una lunghezza variabile:

- **Dimensione minima:** 20 byte (senza il campo Options)
- **Dimensione massima:** 24 byte (con il campo Options)

L'header è allineato a 32 bit (4 byte). Il campo "Length" (IHL - Internet Header Length) indica la lunghezza dell'intestazione misurata in parole di 32 bit (4 byte). Il valore minimo è 5 ($5 \times 4 = 20$ byte), mentre il valore massimo è 6 ($6 \times 4 = 24$ byte), poiché il campo Options occupa fino a 40 byte ma in pratica l'header totale può arrivare a 60 byte se si contano anche le opzioni; il testo del PDF specifica "fino a 24 bytes, con un minimo di 20 bytes" riferendosi alla parte obbligatoria.

11. Quali sono i due componenti fondamentali di un pacchetto IP?

I due componenti fondamentali di un pacchetto IP sono:

1. **L'intestazione (header)** : fornisce le informazioni necessarie per instradare il pacchetto verso la sua destinazione. Include, tra l'altro, l'indirizzo IP di origine, l'indirizzo IP di destinazione, le informazioni

sulla dimensione dell'intero pacchetto, il TTL, il checksum, e altri campi di controllo.

2. **La componente dati (payload)** : contiene il contenuto che viene trasmesso. Può variare in termini di dimensioni e trasporta i dati dei livelli superiori (come segmenti TCP o datagrammi UDP).

Sottosezione 5.2.1 – Pacchetti IPv4

12. Descrivi brevemente la funzione di ciascuno dei seguenti campi dell'header IPv4: Version, Length, Total Length, Identification, Time To Live (TTL), Protocol, Header Checksum.

- **Version**: specifica la versione di IP utilizzata. La versione corrente del protocollo IP è 4 (IPv4). Questo campo permette ai router di interpretare correttamente il formato del datagramma.
- **Length** (IHL - Internet Header Length): indica la lunghezza dell'intestazione del datagramma, misurata in parole di 32 bit (4 byte). Il valore minimo è 5 (20 byte), il massimo è 6 (24 byte). Permette di individuare dove inizia il payload, poiché l'header può avere lunghezza variabile a causa del campo Options.
- **Total Length**: specifica la lunghezza totale del datagramma, comprensiva di intestazione e dati, misurata in byte. È un campo a 16 bit, quindi la lunghezza massima di un datagramma IP è 65.535 byte.
- **Identification**: contiene un intero univoco che identifica il datagramma. Se il datagramma originale viene frammentato, tutti i frammenti avranno lo stesso valore di Identification, permettendo al destinatario di riassemblyli correttamente.
- **Time To Live (TTL)** : è il tempo di vita del pacchetto, per evitare la sua persistenza indefinita sulla rete nel caso in cui non si riesca a recapitarlo al destinatario. Inizialmente misurava i "secondi di vita" del pacchetto, mentre oggi rappresenta il numero di "salti" (hop) da nodo a nodo nella rete. Ogni router che riceve il pacchetto prima di inoltrarlo ne decrementa il valore. Quando TTL arriva a zero, il pacchetto viene scartato e viene inviato un messaggio ICMP al mittente.
- **Protocol**: specifica il tipo di protocollo di livello superiore contenuto nel pacchetto (es. 6 per TCP, 17 per UDP, 1 per ICMP). La lista dei codici di tali protocolli è mantenuta dalla IANA (Internet Assigned Numbers Authority).
- **Header Checksum**: è un campo usato per il controllo degli errori dell'header. Viene calcolato a ogni router e ricalcolato dopo ogni modifica (ad esempio dopo il decremento del TTL). Se il checksum non corrisponde, il pacchetto viene scartato. Questo campo protegge solo l'header, non il payload (i dati), perché i dati sono protetti dai protocolli di livello superiore.

13. A cosa serve il campo Type of Service (ToS) e quali sottocampi contiene?

Il campo Type of Service (ToS) è utilizzato per la Qualità del Servizio (QoS). Contiene cinque sottocampi che specificano il tipo di precedenza, il ritardo, la velocità di trasmissione e l'affidabilità desiderata per quel pacchetto. Le impostazioni predefinite per questi cinque sottocampi sono:

- Precedenza di routine
- Ritardo normale
- Velocità di trasmissione normale

- Affidabilità normale

Questo campo permette a un mittente di richiedere un trattamento differenziato per i propri pacchetti (ad esempio, bassa latenza per il traffico VoIP, alta affidabilità per il trasferimento file). In pratica, il campo ToS è stato poi evoluto in Differentiated Services (DiffServ) con una diversa interpretazione dei bit.

14. Qual è la funzione dei tre bit Flags nell'header IPv4? Spiega il significato di "Don't Fragment" e "More Fragment".

I tre bit Flags sono utilizzati per il controllo del protocollo e della frammentazione dei datagrammi:

- **Bit 0 – Reserved (Reservato)** : sempre settato a 0. Non utilizzato.
- **Bit 1 – Don't Fragment (DF - Non frammentare)** : se settato a 0 (default), la frammentazione è permessa se necessario. Se settato a 1, il pacchetto non viene frammentato. In questo caso, se sulla sua strada incontra un router/host che necessita di frammentarlo (perché la MTU del collegamento successivo è inferiore alla dimensione del pacchetto), il pacchetto viene scartato e viene inviato un messaggio ICMP di errore al mittente. Questo flag è utile per applicazioni che preferiscono perdere un pacchetto piuttosto che riceverlo frammentato (ad esempio perché la frammentazione potrebbe introdurre latenza o complicare il riassemblaggio).
- **Bit 2 – More Fragment (MF - Più frammenti)** : se settato a 0, indica che il pacchetto non è frammentato oppure che è l'ultimo frammento del pacchetto originario. Tutti gli altri frammenti (quelli che non sono l'ultimo) hanno il bit MF settato a 1, così il destinatario sa che deve attendere altri frammenti prima di completare il riassemblaggio.

15. Cosa indica il campo Fragment Offset e in quale unità di misura è espresso?

Il campo Fragment Offset indica lo scostamento (offset) di questo frammento all'interno del datagramma originale. È misurato in unità di **byte**, ma viene espresso in multipli di 8 byte (cioè l'offset è memorizzato in blocchi da 8 byte). Questo permette di rappresentare un offset fino a 65.528 byte utilizzando un campo a 13 bit ($2^{13} = 8192$ blocchi da 8 byte = 65.536 byte totali). Il primo frammento ha offset 0, il secondo frammento ha offset pari alla lunghezza dei dati del primo frammento, e così via. Questo campo, insieme all'Identification e al flag More Fragment, permette al destinatario di riassemblare correttamente i frammenti nell'ordine originale.

16. Come funziona il campo Time To Live (TTL) nell'IPv4 moderno? Cosa rappresenta oggi?

Nel IPv4 moderno, il campo TTL (Time To Live) non rappresenta più il tempo in secondi, ma il **numero di salti (hop) massimi** che il pacchetto può compiere. Il funzionamento è il seguente:

- Il mittente imposta il TTL a un valore iniziale (tipicamente 64, 128, o 255).
- Ogni router che riceve il pacchetto decrementa il TTL di 1 prima di inoltrarlo.
- Se il TTL diventa 0, il router scarta il pacchetto e invia un messaggio ICMP "Time Exceeded" al mittente.

Questo meccanismo impedisce che pacchetti destinati a un host irraggiungibile (o intrappolati in loop di routing) circolino per sempre sulla rete, consumando risorse. Il valore del TTL viene anche utilizzato dal comando **traceroute** per scoprire il percorso dei pacchetti: si inviano pacchetti con TTL crescenti (1, 2, 3, ...) e si raccolgono i messaggi ICMP dai router intermedi.

17. Cosa specifica il campo Protocol nell'header IPv4 e chi mantiene la lista dei codici?

Il campo Protocol specifica il tipo di protocollo di livello superiore (livello trasporto o applicazione) contenuto nel payload del pacchetto IP. Alcuni esempi di codici comuni sono:

- 1 = ICMP (Internet Control Message Protocol)
- 6 = TCP (Transmission Control Protocol)
- 17 = UDP (User Datagram Protocol)

La lista dei codici di tali protocolli è mantenuta dalla **IANA (Internet Assigned Numbers Authority)**, l'ente internazionale che gestisce l'assegnazione dei parametri dei protocolli Internet. Quando un nuovo protocollo viene standardizzato, gli viene assegnato un numero univoco registrato presso la IANA.

Sottosezione 5.2.2 – Indirizzamento IP

18. Come viene rappresentato un indirizzo IPv4 in notazione decimale puntata? Fai un esempio.

Un indirizzo IPv4 in notazione decimale puntata (dot-decimal notation) è rappresentato come quattro numeri decimali separati da punti. Ogni numero corrisponde a un byte (8 bit) dell'indirizzo binario a 32 bit, quindi può assumere valori da 0 a 255.

L'esempio fornito nel PDF è: **208.67.220.220**

Questa notazione testuale è pensata per rendere più semplice la lettura a noi umani. Internamente, gli indirizzi IP vengono memorizzati in formato binario, cioè con una sequenza di 0 e 1.

19. Quanti bit compongono un indirizzo IPv4 e a quanti indirizzi totali corrisponde?

Un indirizzo IPv4 è composto da **32 bit**. La rappresentazione reale viene fatta in binario ed è lunga 32 bit, divisi in 4 ottetti (gruppi di 8 bit).

Il numero totale di indirizzi IPv4 esistenti è 2^{32} , che corrisponde a **4.294.967.296** (circa 4 miliardi e 300 milioni). Tuttavia, il testo specifica che quelli realmente utilizzabili sono un po' di meno in quanto alcuni indirizzi sono riservati (ad esempio 0.0.0.0, 127.0.0.1, 255.255.255.255, e l'intera classe 192.168.0.0/16 degli indirizzi privati).

20. Cos'era l'indirizzamento classfull (a classi)? In quante classi era suddiviso e quali erano?

L'indirizzamento classfull (a classi) era lo schema originario per individuare un indirizzo IPv4. Con questo schema, l'indirizzo è ad autoidentificazione: per capire la classe a cui apparteneva bastava identificare i primi 4 bit dell'IP (i più significativi).

Le classi erano cinque, nominate A, B, C, D, E:

- **Classe A:** primo bit = 0 (indirizzi da 0.0.0.0 a 127.255.255.255). 8 bit per la rete, 24 bit per l'host.
- **Classe B:** primi due bit = 10 (indirizzi da 128.0.0.0 a 191.255.255.255). 16 bit per la rete, 16 bit per l'host.
- **Classe C:** primi tre bit = 110 (indirizzi da 192.0.0.0 a 223.255.255.255). 24 bit per la rete, 8 bit per l'host.
- **Classe D:** primi quattro bit = 1110 (indirizzi da 224.0.0.0 a 239.255.255.255). Riservata per multicast.

- **Classe E:** primi quattro bit = 1111 (indirizzi da 240.0.0.0 a 255.255.255.255). Riservata per usi futuri o sperimentali.

21. In cosa consiste la divisione tra netid e hostid nell'indirizzamento a classi?

Nell'indirizzamento a classi, ogni indirizzo è diviso in due campi:

- **Netid (indirizzo di rete)** : identifica la rete su cui si trova il computer. Costituisce numericamente una specie di prefisso. Tutti i computer sulla stessa rete condividono lo stesso netid.
- **Hostid (indirizzo del computer)** : identifica il computer specifico all'interno della rete. Costituisce numericamente una specie di suffisso. Deve essere univoco all'interno della stessa rete.

La lunghezza di netid e hostid varia a seconda della classe:

- Classe A: netid = 8 bit, hostid = 24 bit
- Classe B: netid = 16 bit, hostid = 16 bit
- Classe C: netid = 24 bit, hostid = 8 bit

22. Quali sono i principali limiti dell'indirizzamento a classi?

I principali limiti dell'indirizzamento a classi sono dovuti soprattutto al numero di host gestibili dalle diverse classi:

- **Rigidità:** le dimensioni fisse delle classi non permettono di allocare reti di dimensione intermedia. Una rete con 300 host, ad esempio, non può usare una classe C (max 254 host) né una classe B (65.534 host, la maggior parte dei quali sarebbero sprecati).
- **Spreco di indirizzi:** le reti di classe B (65.534 host) erano spesso sovradimensionate rispetto alle reali necessità delle organizzazioni, causando un enorme spreco di indirizzi.
- **Esaurimento rapido:** la crescita delle reti LAN negli anni ottanta portò all'esaurimento rapido degli indirizzi disponibili, soprattutto delle classi B e C.
- **Difficoltà di cambiamento:** se si esauriscono gli indirizzi univoci resi disponibili da una classe, occorre fare ricorso a un indirizzo di classe superiore, ma il cambiamento non è indolore (richiede riconfigurazione di tutti i router e host coinvolti).

23. Cosa significa l'acronimo CIDR? Quale innovazione ha introdotto rispetto all'indirizzamento classfull?

CIDR significa **Classless Inter-Domain Routing** (instradamento inter-dominio senza classi). È una tecnica che ha permesso di mantenere in auge IPv4 anche con l'alto numero di dispositivi connessi.

L'innovazione principale introdotta da CIDR consiste nell'utilizzare **maschere di sottorete di lunghezza arbitraria**, al contrario dell'indirizzamento con classi che ne prevedeva solo tre (8, 16, 24 bit). Con CIDR, l'indirizzo IP rimane suddiviso in due parti (rete e host), ma il numero di bit riservati alla rete e quelli dedicati all'host vengono fatti in maniera **dinamica**. In notazione CIDR, si scrive l'indirizzo IP seguito da un carattere slash "/" e dal numero di bit che identificano la rete. Ad esempio, 192.168.0.0/24 indica che i primi 24 bit identificano la rete, i rimanenti 8 bit l'host.

24. Cosa indica la notazione 192.168.0.0/24? Quanti bit identificano la rete e quanti l'host?

La notazione 192.168.0.0/24 è un esempio di rete privata spesso utilizzata in casa in notazione CIDR. Il numero 24 si riferisce al fatto che **i primi 24 bit** (quindi i primi tre ottetti) identificano la **rete**, mentre la parte rimanente (8 bit, cioè l'ultimo ottetto) identifica l'**host**.

- Bit di rete: 24 bit (da 192.168.0)
- Bit di host: 8 bit (l'ultimo numero, da 0 a 255)

Questa rete può quindi ospitare fino a 254 host (256 indirizzi totali, meno l'indirizzo di rete e l'indirizzo di broadcast).

25. Perché il prefisso CIDR non deve essere necessariamente un multiplo di 8?

Il prefisso CIDR non deve essere necessariamente un multiplo di 8 perché la suddivisione tra rete e host è **dinamica e arbitraria**, non vincolata ai confini naturali degli ottetti. Con CIDR, la maschera di sottorete può avere qualsiasi lunghezza, da 1 a 31 bit (o 32 per un singolo host). Ad esempio, un prefisso /26 significa che 26 bit identificano la rete e 6 bit l'host. Questo permette una grana molto più fine nell'allocazione degli indirizzi: si possono creare sottoreti di dimensioni molto varie (ad esempio /30 per collegamenti punto-punto, che hanno solo 2 indirizzi utilizzabili). Il fatto che il prefisso non sia necessariamente un multiplo di 8 rende a volte non immediato capire dove finisce la rete, ma offre una flessibilità molto maggiore.

26. Qual è la differenza tra un indirizzo IP pubblico e uno privato?

La differenza tra indirizzi IP pubblici e privati è la seguente:

- **Indirizzi IP pubblici:** vengono utilizzati quando si interagisce con Internet. Sono univoci a livello globale e devono essere assegnati da un'autorità centrale (IANA o i registri regionali RIR). Un dispositivo con un indirizzo IP pubblico è direttamente raggiungibile da qualsiasi altro dispositivo su Internet.
- **Indirizzi IP privati:** operano solo sulla rete locale (LAN). Non sono univoci a livello globale; lo stesso indirizzo privato può essere utilizzato contemporaneamente in reti diverse senza problemi. I router non inoltrano pacchetti con indirizzi privati su Internet. Per accedere a Internet, questi dispositivi devono passare attraverso un dispositivo che fa NAT (Network Address Translation), tipicamente il router domestico.

Il testo specifica che su una tipica rete con configurazione semplice, il router utilizza un indirizzo IP pubblico per identificarla in maniera univoca sulla rete Internet, mentre all'interno della rete assegna a ciascun dispositivo un indirizzo IP privato (anch'esso univoco all'interno della LAN).

27. Quale dispositivo assegna gli indirizzi IP privati all'interno di una rete domestica?

All'interno di una rete domestica, è il **router** (o altro dispositivo che ne fa le veci, come un modem/router combinato) ad assegnare a ciascun dispositivo sulla rete un indirizzo IP privato (anch'esso univoco all'interno della LAN). Questo avviene tipicamente tramite il protocollo DHCP (Dynamic Host Configuration Protocol), che assegna automaticamente indirizzi IP ai dispositivi che si connettono alla rete. In questo modo, i dati possono essere inviati al dispositivo che li sta effettivamente richiedendo, grazie all'indirizzo privato univoco all'interno della rete locale.

28. Che differenza c'è tra un indirizzo IP statico e uno dinamico? Quali sono i vantaggi di ciascuno?

- **Indirizzo IP statico:** è un tipo di indirizzo IP che non cambia nel tempo. Quando un IP statico viene assegnato a un dispositivo, tale indirizzo rimane tale nel tempo. **Vantaggi:** consente ad alcuni servizi online di funzionare in modo più uniforme e funzionale (ad esempio, un server web o un server FTP può essere sempre raggiunto allo stesso indirizzo). È necessario per dispositivi che devono essere sempre raggiungibili con lo stesso identificatore.
- **Indirizzo IP dinamico:** può cambiare se necessario, come ad esempio quando un dispositivo si connette a una nuova rete. **Vantaggi:** la maggior parte delle reti domestiche utilizza indirizzi IP dinamici perché sono più semplici e più economici da assegnare per i Provider di servizi Internet (uno stesso indirizzo può essere riutilizzato da client diversi in momenti diversi). Inoltre, gli indirizzi dinamici possono risultare più protetti e offrire maggiore anonimato, poiché l'indirizzo cambia periodicamente.

29. Elenca i tre range di indirizzi IP privati (classi A, B e C).

I tre range di indirizzi IP privati (intervalli riservati per l'uso all'interno di reti private) sono:

- **Intervallo IP privato di classe A:** 10.0.0.0 – 10.255.255.255 (un singolo blocco /8, con 16.777.216 indirizzi)
- **Intervallo IP privato di classe B:** 172.16.0.0 – 172.31.255.255 (16 blocchi contigui /16, con 1.048.576 indirizzi)
- **Intervallo IP privato di classe C:** 192.168.0.0 – 192.168.255.255 (256 blocchi /16? in realtà 256 blocchi /24? il testo dice /16, con 65.536 indirizzi. La notazione corretta è 192.168.0.0/16, che copre da 192.168.0.0 a 192.168.255.255)

Il testo specifica che nell'intervallo degli IP privati non importa se lo stesso indirizzo è assegnato anche in altre reti private, poiché essi devono essere univoci solo all'interno della stessa rete locale.

Sezione 5.3 – Protocollo IPv6 e Indirizzamento

30. Perché è stato definito IPv6 e in che anno?

IPv6 è stato definito negli **anni '90** a causa dell'esaurimento degli indirizzi IPv4. La crescita esplosiva di Internet, l'aumento del numero di dispositivi connessi (computer, smartphone, tablet, dispositivi IoT), e l'inefficienza dell'allocazione degli indirizzi IPv4 (dovuta all'indirizzamento a classi e al NAT) hanno reso necessario un nuovo protocollo con uno spazio di indirizzamento molto più ampio. La diffusione di IPv6 è iniziata a metà degli anni 2000, ma trattandosi di un protocollo molto diverso da IPv4 e con retrocompatibilità limitata, la sua crescita è stata molto lenta.

31. Quanti bit ha un indirizzo IPv6 rispetto a IPv4? A quanti indirizzi totali corrisponde?

Mentre IPv4 prevede indirizzi lunghi **32 bit**, in IPv6 essi sono lunghi **128 bit**, cioè il quadruplo. Questo porta ad avere uno spazio di indirizzamento estremamente vasto: si tratta infatti di **2¹²⁸ indirizzi**. Per dare un'idea

della grandezza, 2^{128} è circa $3,4 \times 10^{38}$, un numero così grande da permettere di assegnare miliardi di indirizzi a ogni essere umano sulla Terra.

32. Come viene rappresentato testualmente un indirizzo IPv6? Fai un esempio.

La maggiore lunghezza ha portato a utilizzare il sistema **esadecimale** per la rappresentazione testuale. Un indirizzo IPv6 è composto con un totale di 32 caratteri esadecimali (ogni carattere esadecimale rappresenta 4 bit, quindi $32 \times 4 = 128$ bit) raggruppati in **8 "parole" o gruppi da 4 caratteri ciascuno**, separate dal simbolo due punti ("😊").

L'esempio fornito nel PDF è: **2001:0db8:85a3:0000:0000:8a2e:0370:7334**

33. Come si può comprimere un indirizzo IPv6? Spiega la regola dei leading zeros e della compressione degli zeri.

Un indirizzo IPv6 è effettivamente lungo e difficile da ricordare, ma c'è la possibilità di comprimerlo rimuovendo le parti poco utili. Le regole di compressione sono:

1. **Leading zeros (zeri iniziali)** : si possono rimuovere tutti gli zeri all'inizio di ciascun gruppo. Ad esempio, "0db8" diventa "db8", "0370" diventa "370", "0000" diventa "0".
2. **Compressione degli zeri**: si possono comprimere le parti composte solo da zeri (una serie continua di gruppi con valore 0000) sostituendole con "::". Questa compressione può essere applicata **una sola volta** nello stesso indirizzo, per evitare ambiguità.

Ad esempio, l'indirizzo **2001:0db8:85a3:0000:0000:8a2e:0370:7334** può essere compresso come **2001:db8:85a3::8a2e:370:7334**. Il "::" sostituisce i due gruppi di zeri "0000:0000".

34. Quali sono le tre parti principali della struttura di un indirizzo IPv6 Global Unicast?

Le tre parti principali della struttura dell'indirizzo IPv6 Global Unicast (indirizzo pubblico) sono:

1. **I primi 3 bit** (o i primi 3-16 bit a seconda della definizione): rappresentano il range di indirizzi 'global unicast' allocato da IANA (Internet Assigned Numbers Authority) in questo momento. Attualmente sono allocati gli indirizzi che iniziano per "001" (primo byte=001xxxxx, cioè 2xxx::/3).
2. **Subnet Prefix (la "rete estesa")** : aggiunge al Site Prefix altri 16 bit fissi di sottorete, permettendo fino a 65.000 sottoreti ($2^{16} = 65.536$) per ogni organizzazione o "Site".
3. **Interface ID (l'Host)** : occupa i rimanenti 64 bit (la seconda metà dell'indirizzo, dal bit 65 al bit 128). Identifica la specifica interfaccia di rete del dispositivo (un dispositivo può avere più interfacce, es. WiFi ed Ethernet).

35. Quanti bit occupa l'Interface ID in IPv6 e cosa identifica?

L'Interface ID occupa **64 bit**, cioè la seconda metà dell'indirizzo IPv6 (dal bit 65 al bit 128). Identifica la specifica interfaccia di rete del dispositivo. A differenza di IPv4, dove la parte che separa rete e host è variabile, in IPv6 l'Interface ID ha una lunghezza fissa di 64 bit. Questo è uno dei principi di design di IPv6: semplificare l'indirizzamento avendo una parte host di dimensione fissa e ampiamente sufficiente (2^{64} indirizzi per sottorete, circa 18 miliardi di miliardi).

36. Perché attualmente tutti gli indirizzi IPv6 pubblici iniziano con la cifra 2?

Dalla IANA è stato finora allocato **1/8 dello spazio di indirizzamento unicast IPv6**, e precisamente quello che inizia per "001" (primo byte=001xxxxx, cioè gli indirizzi IPv6 = 2xxx::/3). Quando questo spazio si esaurirà, si passerà a 3xxx::/3, e così via. In pratica, questo significa che attualmente tutti gli indirizzi IPv6 pubblici (global unicast) iniziano con la **cifra 2** (in esadecimale).

37. Come si scrivono in IPv6 l'indirizzo non specificato (unspecified) e l'indirizzo di loopback?

- **Indirizzo non specificato (Unspecified)** : corrisponde in IPv4 allo 0.0.0.0 (cioè "qualsiasi indirizzo"). In IPv6 viene scritto come **::128**. Viene utilizzato tipicamente durante la configurazione iniziale di un dispositivo, quando ancora non ha un indirizzo assegnato.
- **Indirizzo di loopback**: corrisponde al nostro 'localhost', il server presente sulla macchina sulla quale stiamo lavorando. In IPv4 corrisponde al 127.0.0.1. In IPv6 viene indicato come **::1/128**.

38. Cosa sono gli indirizzi link-local in IPv6? Come si riconoscono e a cosa servono?

Gli indirizzi **link-local** sono una categoria di indirizzi IPv6 locali (i corrispondenti degli indirizzi privati in IPv4, ma con ambito più ristretto). Essi vengono sempre bloccati dai Router e sono quindi locali solo a un segmento di rete (switched LAN) o a una subnet. Non vengono instradati verso l'esterno.

Come si riconoscono: iniziano con i 9 bit "11111110 1" (FE8x::/9 fino a FEBx::/9? Il testo specifica: hanno come decimo bit uno "0", per cui cominciano con FE8x, FE9x, FEAx e FEBx). In notazione compressa, gli indirizzi link-local iniziano con **fe80::/10**.

A cosa servono: vengono usati per:

- La "automatic address configuration" (SLAAC)
- Le funzioni ND (Neighbor Discovery), che sostituiscono l'ARP di IPv4
- La comunicazione tra dispositivi sulla stessa subnet senza bisogno di indirizzi globali

39. Cosa sono gli indirizzi site-local in IPv6? Come si differenziano dai link-local?

Gli indirizzi **site-local** sono un'altra categoria di indirizzi IPv6 locali. Possono essere instradati dai Router di una organizzazione solo all'interno della rete privata (Site), quindi tra le sue subnet, ma **non verso Internet**.

Come si riconoscono: iniziano con FECx, FEDx, FEEx ed FEFx, avendo come decimo bit un "1" (i link-local hanno decimo bit = 0).

Differenza rispetto ai link-local:

- **Link-local:** ambito limitato a un singolo segmento di rete (non attraversano router)
- **Site-local:** ambito limitato a un intero sito/azienda (possono essere instradati all'interno dell'organizzazione, ma non verso Internet)

Il testo specifica che gli indirizzi site-local sono stati poi deprecati in favore degli Unique Local Addresses (ULA), ma il concetto rimane valido per comprendere la gerarchia degli indirizzi IPv6.

40. Qual è la differenza fondamentale tra IPv4 e IPv6 per quanto riguarda la necessità del NAT?

La differenza fondamentale è che in IPv6 l'idea è che ciascun dispositivo abbia assegnato **almeno un indirizzo pubblico visibile su Internet** (global unicast). Per questo motivo, un indirizzo di questo tipo è definito **global unicast (GUA)**. In IPv4, invece, a causa della penuria di indirizzi, molti dispositivi sono costretti a usare indirizzi privati e ricorrere all'ausilio del **NAT (Network Address Translation)** per comunicare con Internet.

In IPv6, lo spazio di indirizzi è così vasto che non è necessario il NAT. Ogni dispositivo può avere il proprio indirizzo pubblico univoco, semplificando l'architettura di rete e ripristinando il principio di connettività end-to-end che in IPv4 era stato compromesso proprio dal NAT. Gli indirizzi privati in IPv6 (come i link-local e i site-local) sono ancora utili per servizi che devono rimanere interni (ad esempio nelle reti aziendali), ma non sono una necessità imposta dalla scarsità di indirizzi.

Sottosezione 5.3.1 – Prefix Delegation e Subnetting

41. Cosa significa Prefix Delegation in IPv6? Quale protocollo lo implementa?

La **Prefix Delegation** (delega di prefisso) è un meccanismo che permette a un router dell'operatore (ISP) di delegare al router del cliente la gestione di un prefisso IPv6, consentendogli di creare sottoreti e assegnare indirizzi pubblici a tutti i dispositivi in esse presenti. In pratica, l'ISP non assegna un singolo indirizzo al cliente, ma un intero blocco di indirizzi (un prefisso) che il cliente può suddividere come preferisce.

Il protocollo che implementa questa funzionalità è **DHCPv6** (Dynamic Host Configuration Protocol versione 6), con l'estensione per la Prefix Delegation.

42. Secondo le raccomandazioni europee, quali prefissi dovrebbero essere delegati dagli ISP?

Secondo le raccomandazioni europee, gli ISP dovrebbero delegare un prefisso **statico /56** oppure **/48** a ciascuna linea. Questi prefissi prenderebbero la forma di:

- 2001:db8:aaaa::/48 (indirizzo di esempio)
- 2001:db8:aaaa:1a00::/56 (indirizzo di esempio)

Un prefisso /48 è più grande (16 bit di subnet disponibili) ed è tipico per utenti aziendali o residenziali con molte esigenze di subnetting, mentre /56 è leggermente più piccolo ma ancora ampiamente sufficiente per la maggior parte degli utenti.

43. Dato un prefisso /56, quante sottoreti /64 si possono ottenere? Perché?

Dato un prefisso /56, si possono ottenere **256 sottoreti /64**. Il calcolo è: $2^{64-56} = 2^8 = 256$.

Questo perché l'Interface ID in IPv6 occupa sempre 64 bit. I bit restanti ($64 - 56 = 8$ bit) sono i bit che possono essere variati per creare sottoreti diverse. Quindi, con un prefisso /56 delegato dall'ISP, il cliente può creare fino a 256 sottoreti diverse, ciascuna con il proprio spazio di indirizzi /64 (che a sua volta contiene 2^{64} indirizzi, un numero astronomico).

44. Cosa significa l'acronimo SLAAC? A cosa serve?

SLAAC significa **Stateless Address Auto-Configuration** (autoconfigurazione stateless degli indirizzi). È un sistema che permette a tutti i dispositivi che supportano IPv6 di disporre in automatico di un indirizzo link-local e di auto-configurarsi assegnandosi un indirizzo IPv6 senza bisogno di un server DHCP.

Il funzionamento è il seguente: attraverso l'indirizzo link-local e le funzioni di Neighbor Discovery (ND), il dispositivo scopre com'è fatta la rete in cui si trova (quale prefisso è annunciato dai router locali) e genera autonomamente la propria parte Interface ID, ottenendo così un indirizzo IPv6 completo. SLAAC non richiede di mantenere una lista di IP assegnati come farebbe un server DHCPv6.

45. Quali sono le due procedure per generare un indirizzo auto-assegnato con SLAAC?

Le due procedure per generare l'Interface ID con SLAAC sono:

1. **Assegnazione casuale (random)** : come dice il nome stesso del processo, l'indirizzo IPv6 (l'Interface ID) viene generato in modo casuale. Questo approccio offre maggiore privacy perché l'indirizzo non è legato in modo permanente all'hardware del dispositivo.
2. **Sistema EUI-64**: l'Interface ID viene generato a partire dall'indirizzo MAC (Media Access Control) della scheda di rete, che è un indirizzo a 48 bit. L'IPv6 EUI-64 inserisce i due byte "ff:fe" al centro dell'indirizzo MAC (dopo i primi 3 byte). Un indirizzo IPv6 generato con EUI-64 è facilmente riconoscibile perché contiene sempre al centro i caratteri **ff:fe** (ad esempio, da MAC 00:11:22:33:44:55 si ottiene Interface ID 0211:22ff:fe33:4455).

46. Cosa sono le Privacy Extensions in IPv6 e a cosa servono?

Le **Privacy Extensions** (estensioni per la privacy) sono una funzionalità implementata nelle versioni più recenti dei sistemi operativi. Esse consistono nel **cambiare periodicamente l'indirizzo auto-configurato** (quello generato con SLAAC) per rendere più difficile il tracciamento e mantenere la privacy.

Senza queste estensioni, se si usa EUI-64, l'indirizzo IPv6 di un dispositivo contiene permanentemente l'indirizzo MAC, che è un identificatore unico legato all'hardware. Questo permetterebbe di tracciare il dispositivo ovunque si connetta. Con le Privacy Extensions, il dispositivo genera indirizzi temporanei casuali che cambiano nel tempo (tipicamente ogni poche ore o giorni), rendendo molto più difficile correlare le attività dello stesso utente attraverso sessioni diverse.

Sottosezione 5.3.2 – Altre Novità di IPv6

47. Quali campi sono stati rimossi dall'header IPv6 rispetto a IPv4? Perché?

Nell'header IPv6 è stato **rimosso il campo checksum**. La motivazione è semplificare e velocizzare l'inoltro dei pacchetti sui router. In IPv4, ogni router deve ricalcolare il checksum dell'header dopo aver decrementato il TTL, un'operazione che richiede tempo di calcolo. In IPv6, eliminando il checksum, i router possono inoltrare i pacchetti più velocemente, poiché non devono eseguire questo calcolo. L'affidabilità end-to-end è comunque garantita dai checksum dei protocolli di livello superiore (TCP, UDP, etc.) e dai livelli di collegamento.

Sono stati rimossi anche altri campi come il campo "Length" (sostituito da un campo "Payload Length" più semplice), i campi di frammentazione (spostati in extension header opzionali), e ovviamente l'indirizzamento broadcast (sostituito dal multicast).

48. Quanto è lungo l'header IPv6 in byte? Perché è più lungo di quello IPv4?

L'header IPv6 è lungo **40 byte** (fissi). È più lungo dell'header IPv4 (20-24 byte) principalmente a causa degli **indirizzi più lunghi**: mentre IPv4 usa indirizzi a 32 bit (4 byte per sorgente e 4 byte per destinazione, totale 8 byte), IPv6 usa indirizzi a 128 bit (16 byte per sorgente e 16 byte per destinazione, totale 32 byte). La differenza di 24 byte negli indirizzi da sola spiega gran parte della differenza di lunghezza tra i due header (40 vs 20).

49. Cosa sono gli Extension Header in IPv6?

Gli **Extension Header** (intestazioni estese) sono una novità di IPv6. L'header IPv6 è anche estensibile, cioè permette di definire funzionalità aggiuntive inserendo header "a catena" come contenuto del pacchetto. Invece di avere campi opzionali all'interno dell'header principale (come il campo Options di IPv4), IPv6 definisce header separati che vengono concatenati uno dopo l'altro.

Ogni Extension Header è un intero **multiplo di 8 ottetti (8 byte)** per permettere l'allineamento naturale dei successivi. Devono essere processati in modo sequenziale dal router o dall'host destinatario. Esempi di extension header includono: header per la frammentazione, header per il routing (source routing), header per l'autenticazione (AH), header per il payload di sicurezza (ESP).

50. Quali tre macro-categorie di indirizzi sono previste in IPv6? Quale categoria è stata eliminata rispetto a IPv4?

IPv6 prevede tre macro-categorie di indirizzi:

1. **Unicast**: indirizzo singolo, identifica una singola interfaccia
2. **Anycast**: identifica un insieme di interfacce (di solito su nodi diversi). I pacchetti inviati a un indirizzo anycast vengono consegnati a una sola di esse (la più vicina in termini di routing)
3. **Multicast**: identifica un gruppo di interfacce; i pacchetti vengono consegnati a tutte le interfacce del gruppo

La categoria **eliminata rispetto a IPv4** è il **broadcast**. In IPv4, il broadcast permetteva di inviare un pacchetto a tutte le interfacce di una rete. In IPv6, la stessa funzionalità è ottenuta utilizzando indirizzi multicast specifici (ad esempio, il multicast "tutti i nodi" sostituisce il broadcast).

51. A cosa serve un indirizzo multicast in IPv6? Quale prefisso hanno?

Un indirizzo **multicast** in IPv6 viene attribuito a un gruppo di interfacce di rete differenti (in linea di principio, ognuna di esse appartenente ad altrettanti dispositivi distinti). I pacchetti inviati a un indirizzo multicast sono diretti a **tutte le interfacce** che fanno riferimento a questo indirizzo, ossia a quelle alle quali è stato attribuito.

Il prefisso per gli indirizzi multicast è **ff00::/8** (primo byte = 0xff). Il secondo byte del prefisso specifica le caratteristiche dell'indirizzo multicast (flag e scope, cioè l'ambito di propagazione: nodo locale, link locale, sito locale, organizzazione, globale).

52. Cosa sono gli indirizzi anycast? Come si comporta la rete quando viene inviato un pacchetto a un indirizzo anycast?

Gli indirizzi **anycast** hanno le stesse caratteristiche di quelli unicast, ma vengono attribuiti a diverse interfacce di altrettanti nodi. Lo scopo di un indirizzo anycast è quello di far sì che la rete consegni le informazioni al **nodo del gruppo che risponde prima** (ad esempio quello più vicino in base al protocollo di instradamento). Per la precisione, i pacchetti inviati a un indirizzo anycast dovrebbero raggiungere **un'unica interfaccia di rete** (la migliore in termini di costo, tipicamente la più vicina in termini di routing).

Questo meccanismo è utile per servizi replicati su più server: ad esempio, i root DNS server hanno tutti lo stesso indirizzo anycast; un client viene automaticamente indirizzato al server fisicamente più vicino.

53. Quali due extension header forniscono sicurezza in IPv6? Che differenza c'è tra AH e ESP?

I due extension header che forniscono sicurezza in IPv6 sono:

1. **IP Authentication Header (AH)** : ha il compito di fornire ai datagrammi IP **integrità e autenticazione**, ma **non confidenzialità** (questo per mantenere compatibilità con ambienti dove l'uso della crittografia è limitato). AH rende sicura la comunicazione tra due o più host, tra due o più gateway, o tra host e gateway. AH protegge l'intero pacchetto (header e dati) tranne alcuni campi che cambiano durante il transito.
2. **IP Encapsulation Security Payload (ESP)** : a differenza del precedente, questa modalità fornisce **integrità, autenticazione e anche confidenzialità** ai pacchetti (attraverso la crittografia dei dati). Come la precedente, può essere usata per la comunicazione sicura tra due o più host, tra due o più gateway, o tra host e gateway.

La differenza fondamentale è che ESP offre **confidenzialità** (crittografia del payload), mentre AH no. In pratica, AH è stato in gran parte sostituito da ESP, che può opzionalmente fornire anche autenticazione senza crittografia.

54. Qual è la differenza tra Transport Mode e Tunnel Mode per AH e ESP?

Sia AH che ESP possono essere utilizzati in due modalità:

- **Transport Mode:** viene fornita protezione **solo ai dati** (al payload del pacchetto IP). L'header IP originale rimane in chiaro. Questa modalità è tipicamente usata per comunicazioni host-to-host (end-to-end), dove gli host stessi sono i punti terminali della comunicazione sicura.
- **Tunnel Mode:** viene fornita protezione **all'intero datagramma IP** (sia header che dati). L'intero pacchetto originale viene incapsulato (cifrato) e viene aggiunto un nuovo header IP esterno. Questa modalità è tipicamente usata per gateway VPN (Virtual Private Network), dove i gateway creano un "tunnel" sicuro tra due reti, e i pacchetti originali sono completamente protetti.

55. Cosa si intende per Security Association (SA)?

Una **Security Association (SA)** è un concetto fondamentale sia per AH che per ESP. Una SA è una **relazione, fra due entità in comunicazione, che identifica particolari condizioni di sicurezza**. In pratica, una SA definisce un accordo unidirezionale tra due comunicanti che specifica:

- Quale algoritmo di autenticazione utilizzare
- Quale algoritmo di crittografia utilizzare (per ESP)

- Le chiavi segrete condivise
- Un identificatore univoco (Security Parameters Index, SPI)

Poiché la SA è unidirezionale, una comunicazione bidirezionale richiede due SA (una per ogni direzione). Le SA sono gestite dal protocollo IKE (Internet Key Exchange).

Sezione 5.4 – Algoritmi di Routing

56. Quali sono i due servizi principali forniti da un router?

Secondo il testo, il compito del router è fornire due servizi principali:

1. **Determinare il percorso ottimale** (path determination): il router deve calcolare, utilizzando gli algoritmi di routing, il percorso migliore per raggiungere ogni destinazione, tenendo conto di metriche come il numero di hop, il costo, la banda disponibile, il ritardo, ecc.
2. **Trasportare le informazioni tra 2 reti diverse** (packet forwarding): una volta determinato il percorso, il router deve effettivamente inoltrare i pacchetti ricevuti su una delle sue interfacce verso la destinazione successiva, consultando la propria tabella di routing.

57. Cosa si intende per Autonomous System (AS)? A cosa serve l'ASN?

Un **Autonomous System (AS)** è un gruppo di network controllati e gestiti da fornitori di servizio Internet (ISP). In altre parole, un AS è una rete individuale su Internet. I sistemi autonomi sono di proprietà dei fornitori di servizi Internet.

L'**ASN (Autonomous System Number)** è un numero intero che identifica univocamente ciascun AS. Viene assegnato dalla stessa autorità che rilascia gli indirizzi Internet (IANA e i registri regionali RIR). L'ASN consente a un AS di comunicare sia esternamente che internamente (in questo caso usa un ASN privato). I router che instradano i messaggi tra AS diversi utilizzano protocolli di routing esterni (EGP, come BGP), mentre quelli che instradano all'interno dello stesso AS utilizzano protocolli di routing interni (IGP, come OSPF o RIP).

58. Qual è la differenza tra interior router e edge router?

- **Interior router (router interni)** : sono i router che instradano i messaggi all'interno dello stesso AS. Scambiano le informazioni tramite un protocollo (protocollo IGP - Interior Gateway Protocol) che è uguale su tutti i router interni all'AS.
- **Edge router (router di frontiera)** : sono i router che instradano i messaggi tra AS diversi. Sono anche detti "router di frontiera". Utilizzano protocolli di routing esterni (EGP - Exterior Gateway Protocol, come BGP) per scambiare informazioni di raggiungibilità con altri AS.

59. Quali due elementi deve contenere come minimo ogni entry di una tabella di routing?

Come minimo, ogni entry (voce) nella tabella di routing deve contenere due elementi:

1. **Un indirizzo di destinazione:** l'indirizzo IP del nodo di destinazione o della rete che lo ospita. Questo è l'identificatore della destinazione a cui inoltrare i pacchetti.

2. **Il tipo d'indirizzo di destinazione:** può indicare che la destinazione è direttamente collegata al router oppure può indicare l'indirizzo di un altro router, anche questo collegato direttamente alla rete (next-hop router), che potrebbe portare a ulteriori reti.

In altre parole, ogni entry specifica "dove voglio andare" (indirizzo destinazione) e "da che parte andare" (interfaccia di uscita e/o next-hop).

60. Qual è la differenza tra algoritmi di routing statici e dinamici?

- **Algoritmi di routing statici:** in essi, le tabelle di routing che vengono memorizzate sono predisposte da una persona (Network Administrator) e i valori di tali tabelle **non cambiano** per nessun motivo fino a quando il Network Administrator non li cambia manualmente. Sono semplici da implementare ma non si adattano ai cambiamenti della rete (guasti, nuove connessioni, variazioni di carico).
- **Algoritmi di routing dinamici:** le tabelle vengono **continuamente aggiornate e cambiate** seguendo l'evoluzione della rete (caduta/inserimento di un nodo o di una network). I router scambiano informazioni sullo stato della rete e calcolano automaticamente nuovi percorsi quando necessario. Sono più complessi ma si adattano automaticamente ai cambiamenti.

61. Quali metriche possono essere utilizzate per selezionare il percorso migliore? Quali due sono universalmente accettati?

Per selezionare il percorso migliore, gli algoritmi di routing possono scegliere secondo diversi criteri (metriche), tra cui:

- Lunghezza del percorso (numero di hop)
- Banda passante
- Affidabilità del link
- Ritardo (latenza)
- Carico di rete

Tuttavia, due parametri **universalmente accettati** sono:

1. **Hops:** numero di salti effettuati, cioè il numero di nodi attraversati lungo il cammino. È una metrica semplice che conta quanti router si attraversano.
2. **Costo:** somma dei costi di tutte le linee attraversate. Il costo di una linea è inversamente proporzionale alla sua velocità (linee più veloci hanno costo minore). Per costo si può intendere una qualsiasi delle metriche sopra elencate (banda, ritardo, affidabilità, ecc.), o una combinazione di esse.

62. Quali sono le cinque caratteristiche tipiche degli algoritmi di routing (ottimizzazione, semplicità, rapidità di convergenza, scalabilità, robustezza)?

Le cinque caratteristiche tipiche degli algoritmi di routing sono:

1. **Ottimizzazione:** l'abilità dell'algoritmo a scegliere la strada migliore (minimizzazione o massimizzazione della metrica scelta, ad esempio minimizzare il numero di hop o il costo).
2. **Semplicità:** l'algoritmo deve essere funzionale ed efficiente con basso utilizzo delle risorse hardware (CPU, memoria). Un algoritmo troppo complesso potrebbe rallentare il router.

3. **Rapidità di convergenza:** quando avviene un cambiamento nella rete (es. un link o un router si guasta), i routers devono distribuire, nel più breve tempo possibile, i messaggi di aggiornamento di tale evento affinché non si verifichino malfunzionamenti (loop, percorsi errati, perdita di pacchetti).
4. **Scalabilità:** i routers devono adattarsi velocemente e accuratamente a una varietà di circostanze quali per esempio la caduta di una network o l'aggiunta di molti nuovi router. L'algoritmo deve funzionare bene sia in reti piccole che in reti molto grandi.
5. **Robustezza:** a fronte di guasti hardware, uso intensivo delle risorse hardware e traffico elevato, l'algoritmo deve continuare a lavorare correttamente. Non deve collassare o comportarsi in modo imprevedibile in situazioni anomale.

Sottosezione 5.4.1 – Distance Vector

63. Su quale algoritmo matematico si basa il Distance Vector?

L'algoritmo Distance Vector si basa sull'**algoritmo di Bellman-Ford**. Questo algoritmo matematico permette di calcolare i percorsi minimi in un grafo in presenza di pesi (costi) sugli archi, anche quando alcuni costi sono negativi (anche se in routing generalmente si usano costi positivi). Bellman-Ford viene eseguito in modo distribuito: ogni nodo comunica periodicamente ai suoi vicini la propria conoscenza delle distanze verso tutte le destinazioni.

64. Qual è l'idea base dell'algoritmo Distance Vector?

L'idea base dell'algoritmo Distance Vector è che **ogni nodo della rete mantenga al proprio interno una tabella che ne indica la distanza da ogni altro nodo della rete** (il vettore delle distanze, per l'appunto). Ogni nodo conosce la distanza (costo) per raggiungere i propri vicini diretti. Periodicamente, ogni nodo invia ai suoi vicini una copia del proprio vettore delle distanze. Quando un nodo riceve un vettore di distanze da un vicino, aggiorna la propria tabella di routing se scopre un percorso migliore (a costo minore) verso una destinazione passando per quel vicino. Questo processo iterativo porta tutti i nodi a conoscere le distanze minime verso tutte le destinazioni.

65. Cosa contiene la tabella di routing di un nodo al "cold start"?

Al momento del "cold start" (accensione contemporanea di tutti i nodi), la rete viene inizializzata e ogni nodo è caratterizzato da una conoscenza locale, ossia esso sa solamente il proprio indirizzo e ignora totalmente la topologia della rete e la distanza dagli altri. La tabella di routing sarà quindi **minima, caratterizzata da una singola voce: quella del nodo stesso**. L'esempio fornito nel PDF mostra una rete composta da 3 nodi (chiamati A, C, D) – in realtà l'esempio originale aveva A, B, C – e il nodo A al cold start contiene solo la voce per se stesso.

66. Cosa succede in caso di malfunzionamento di un link? Quali tre azioni compie il nodo?

In caso di malfunzionamento su uno o più link, il nodo compie tre azioni:

1. **Scarta tutti i distance vector ricevuti da quel link** (il link guasto) – non tiene più conto delle informazioni provenienti da quella direzione, poiché il collegamento non è più affidabile.
2. **Ricalcola la propria tabella mediante la fusione dei distance vector** – combina i vettori ricevuti dai link ancora funzionanti per ottenere una nuova stima delle distanze verso tutte le destinazioni.

3. **Distribuisce eventualmente il suo nuovo distance vector ai propri vicini** – se la ricalcolazione ha prodotto cambiamenti significativi, il nodo informa i suoi vicini (attraverso i link ancora funzionanti) della nuova situazione, in modo che possano a loro volta aggiornare le loro tabelle.

67. Cosa sono l'effetto rimbalzo (bouncing) e il counting to infinity?

- **Effetto rimbalzo (bouncing)** : è un fenomeno che si verifica a causa di un possibile tempo di vulnerabilità tra l'indisponibilità del link e l'invio del distance vector del nodo che non sarà più disponibile. Durante questo intervallo di tempo, i nodi vicini potrebbero ancora credere che il link sia funzionante e continuare a inviare pacchetti verso la destinazione ormai irraggiungibile, causando l'aggiornamento delle tabelle che inizia a far divergere l'algoritmo. L'algoritmo si ferma solo al giungere del TTL (Time To Live) dei pacchetti.
- **Counting to infinity (conteggio all'infinito)** : è un problema che si manifesta quando la caduta di più link rende effettivamente isolati i nodi. In questo caso, non vi è alcuna possibilità per l'algoritmo di convergere ad uno stato stabile. I nodi continuano a scambiarsi informazioni che fanno aumentare progressivamente la distanza stimata verso le destinazioni irraggiungibili, fino a raggiungere un valore convenzionale di "infinito". Questo processo può essere terminato solo attraverso una convenzione sulla rappresentazione di infinito con una distanza settata ad un valore maggiore del diametro della rete (ossia del percorso più lungo presente nella rete). Quando la distanza raggiunge questo valore, l'entry nella tabella di routing viene considerata infinitamente distante e, di conseguenza, irraggiungibile.

68. Come si può terminare il counting to infinity?

Il counting to infinity può essere terminato solo attraverso una **convenzione sulla rappresentazione di infinito** con una distanza settata ad un valore maggiore del diametro della rete (ossia del percorso più lungo presente nella rete). Quando la distanza raggiunge questo valore, l'entry nella tabella di routing viene considerata infinitamente distante e, di conseguenza, irraggiungibile.

In pratica, si sceglie un valore massimo (ad esempio 16 in RIP) che rappresenta "infinito". Quando una metrica raggiunge questo valore, si considera che la destinazione non è raggiungibile. Questo limita l'effetto del counting to infinity perché la distanza non può crescere oltre quel valore. Tuttavia, se il diametro della rete è maggiore del valore scelto, si rischiano di classificare come irraggiungibili destinazioni che in realtà sono raggiungibili.

69. In cosa consiste la tecnica Split Horizon?

La tecnica **Split Horizon** (orizzonte diviso) è una delle tecniche principali per minimizzare l'effetto rimbalzo e il counting to infinity. Si basa su una precauzione molto semplice: se un nodo (A) sta instradando pacchetti per una destinazione (Z) attraverso un nodo intermedio (B), non è logico che quest'ultimo (B) tenti di raggiungere (Z) attraverso A. Quindi non c'è ragione per cui (A) debba annunciare a (B) che (Z) ha una distanza più breve da A.

Nella pratica, ciò si realizza facendo in modo che i nodi inviino **differenti versioni del vettore** (per tenere conto del fatto che alcune destinazioni sono instradate attraverso i suddetti links in uscita) invece di diffondere lo stesso distance vector su tutte le porte del router. In altre parole, un nodo non annuncia a un vicino le rotte che ha appreso attraverso quel vicino stesso.

70. Come funziona la variante più aggressiva "Split Horizon con poison reverse"?

La versione di "**Split Horizon con poison reverse**" (orizzonte diviso con rovescio avvelenato) è più aggressiva rispetto allo Split Horizon base. In questa variante:

- Il vettore delle distanze dei nodi continua a includere **tutte le destinazioni raggiungibili** (non si escludono le destinazioni instradate attraverso un dato link).
- Tuttavia, essi **imposteranno a infinito** quelle distanze che sono instradate attraverso il link (quello su cui il vettore viene inviato).
- Non si limitano cioè a escludere dal vettore le entries che sono per loro irraggiungibili, ma le includono con costo infinito, evitando di fatto che altri nodi possano erroneamente 'credere' di poter raggiungere quelle destinazioni per vie alternative.

Il "veleno" (poison) è l'informazione che la rotta è a costo infinito, che scoraggia il vicino dall'usare quella direzione. In questo modo, se un link si guasta, la propagazione di informazioni errate viene rapidamente interrotta.

Sottosezione 5.4.2 – Link State

71. Qual è il principio base degli algoritmi Link State?

Il principio base degli algoritmi Link State (stato dei collegamenti) è diverso da quello del Distance Vector: questi algoritmi **non calcolano in maniera distribuita i percorsi migliori**, bensì **ogni nodo mantiene una copia intera della mappa di rete**, scambiando con ogni altro nodo della rete le informazioni sui suoi vicini, ed esegue localmente un calcolo dei migliori percorsi per raggiungere gli altri nodi (tipicamente con l'algoritmo di Dijkstra).

In altre parole, invece di scambiarsi vettori di distanze, i nodi si scambiano informazioni sulla **topologia** della rete (quali nodi sono connessi e con quale costo). Ogni nodo costruisce una mappa completa della rete e poi calcola da solo i percorsi minimi.

72. Quali sono i tre concetti chiave del Link State Routing?

I tre concetti chiave per capire il Link State Routing sono:

1. **Conoscenza dei nodi vicini:** al posto di inviare la propria tabella di routing, un router invia solo le informazioni sui suoi vicini. Ciascun router perciò invia a tutti i nodi della rete la sua identità e i 'pesi' dei link che lo collegano direttamente ai suoi vicini.
2. **Flooding (inondazione)** : ogni router invia le informazioni che detiene a **tutti** gli altri router della rete tranne a quelli a esso direttamente connessi. Ogni router che riceve il pacchetto di informazione ne invia una copia ai propri vicini (a sua volta, tranne a quello da cui l'ha ricevuto). A regime, ogni router riceve una copia contenente le stesse informazioni.
3. **Condivisione delle informazioni:** un router invia informazioni a ogni altro router **solo quando vi è un cambiamento** che modifica lo stato dei suoi link. Questo rende il protocollo efficiente perché non si inviano aggiornamenti periodici superflui.

73. Cosa si intende per flooding nel contesto del Link State?

Nel contesto del Link State, il **flooding** (inondazione) è il processo mediante il quale ogni router invia le informazioni che detiene a tutti gli altri router della rete tranne a quelli a esso direttamente connessi. Il funzionamento è il seguente:

1. Un router crea un pacchetto contenente le informazioni sui propri link (i vicini e i costi).
2. Invia una copia di questo pacchetto a tutti i suoi vicini.
3. Ogni vicino, a sua volta, inoltra il pacchetto a tutti i suoi vicini (tranne a quello da cui l'ha ricevuto).
4. Il processo continua fino a quando tutti i router della rete hanno ricevuto il pacchetto.

Il flooding garantisce che le informazioni sullo stato dei link si diffondano a tutta la rete in modo affidabile e (relativamente) veloce, anche se non è il metodo più efficiente in termini di messaggi scambiati.

74. Quando un router invia informazioni agli altri router in un protocollo Link State?

Un router invia informazioni a ogni altro router **solo quando vi è un cambiamento che modifica lo stato dei suoi link**. Ad esempio, quando:

- Un link si guasta o viene ripristinato
- Il costo di un link cambia
- Un nuovo router viene aggiunto alla rete

Questo approccio è diverso dai protocolli Distance Vector, che tipicamente inviano aggiornamenti periodici anche in assenza di cambiamenti. L'approccio "solo su cambiamento" (event-driven) riduce il traffico di controllo in condizioni di stabilità, ma richiede meccanismi robusti per garantire che tutti i router ricevano le informazioni in caso di perdita di messaggi.

75. Quali sono le quattro fasi dell'algoritmo Link State?

L'algoritmo Link State può essere suddiviso in quattro fasi:

1. **Inizializzazione:** ogni router della rete 'si informa' sui link che sono ad esso direttamente connessi. Le informazioni così apprese vengono inserite nella tabella del router.
2. **Costruzione e distribuzione dei pacchetti:** ogni router crea un pacchetto (Link State Packet, LSP) contenente la lista dei suoi vicini e i rispettivi costi di collegamento verso ciascuno di loro. Nell'intestazione del pacchetto, ogni router aggiunge la propria identità insieme a un numero di sequenza (per identificare le versioni più recenti) e un valore di durata (age). Il pacchetto viene quindi inviato nella rete tramite **flooding**, per raggiungere ogni nodo.
3. **Calcolo del percorso minimo:** ogni router usa le informazioni appena ricevute (la mappa completa della rete) per creare un 'sink tree' e trovare il percorso minimo verso ogni nodo della rete. Tipicamente viene applicato l'**algoritmo di Dijkstra** per calcolare tali percorsi.
4. **Consolidamento delle route:** in quest'ultima fase, i percorsi calcolati vanno a formare la tabella che poi verrà salvata all'interno del dispositivo. La tabella è formata da tutti i percorsi ottimi verso ogni nodo della rete.

76. Quale algoritmo viene tipicamente usato nella fase 3 per calcolare il percorso minimo?

Nella fase 3 (calcolo del percorso minimo) viene tipicamente applicato l'**algoritmo di Dijkstra**. Questo algoritmo, dato un grafo con pesi non negativi sugli archi, calcola il percorso a costo minimo da un nodo

sorgente a tutti gli altri nodi del grafo. Nel contesto del Link State, ogni router si considera come sorgente e applica Dijkstra alla mappa completa della rete che ha appreso, ottenendo i percorsi minimi verso tutte le destinazioni.

77. Quali sono i principali vantaggi degli algoritmi Link State rispetto a Distance Vector?

I principali vantaggi degli algoritmi Link State sono:

1. **Convergenza abbastanza veloce:** poiché ogni nodo ha bisogno di sapere solo lo stato dei link verso i nodi adiacenti, e le informazioni vengono propagate tramite flooding, gli aggiornamenti si diffondono rapidamente. Questo è molto importante nelle reti molto estese e con alto tasso di cambiamento di topologia.
2. **Non soffre della problematica della divergenza (counting to infinity) :** poiché le informazioni che si scambiano sono lo stato dei link circoscritti ai propri vicini, non esiste il problema del conteggio all'infinito tipico del Distance Vector. Ogni nodo ha una visione completa e coerente della topologia (o almeno la costruisce in modo indipendente).

78. Quali sono i principali svantaggi degli algoritmi Link State?

I principali svantaggi degli algoritmi Link State sono:

1. **Difficile implementazione in reti molto vaste:** non è scalabile come la famiglia del Distance Vector. In una rete con molti nodi, la quantità di informazioni da scambiare (flooding) e da memorizzare (la mappa completa della rete) può diventare proibitiva.
2. **Richiede un uso intensivo di risorse (memoria e tempo di CPU) :** ogni router deve memorizzare l'intera mappa di rete (che può essere molto grande in reti vaste) e deve eseguire l'algoritmo di Dijkstra (che ha complessità $O(n^2)$ o superiore) ogni volta che la topologia cambia. Questo richiede risorse computazionali significative, specialmente in reti di grandi dimensioni.

79. Cosa significa l'acronimo OSPF? Quali sono le tre versioni e cosa le caratterizza?

OSPF significa **Open Shortest Path First** (prima il percorso più breve aperto). È uno dei più importanti algoritmi appartenenti alla famiglia Link State.

Le tre versioni di OSPF sono:

1. **OSPFv1:** la prima versione, che ha gettato le basi per questo algoritmo molto usato, creata nel 1989, ma ormai quasi del tutto scomparsa nella realtà.
2. **OSPFv2:** in vigore dal 1998, usata nelle reti che hanno in uso IPv4. È la versione più comune ancora oggi in molte reti.
3. **OSPFv3:** l'ultima versione di OSPF, creata nel 2008, introduce l'uso di IPv6 pur rimanendo compatibile a IPv4. OSPFv3 è un aggiornamento sostanziale che supporta IPv6 nativamente, anche se può trasportare anche informazioni IPv4.

80. Cosa sono gli LSA (Link State Advertisements) e l'LSDB (Link State Database)?

- **LSA (Link State Advertisements)** : sono i pacchetti che i router OSPF utilizzano per immagazzinare e scambiare le informazioni riguardanti la rete. Ogni LSA contiene informazioni su una parte della topologia (ad esempio, i link di un router e i loro costi). I diversi tipi di LSA servono per scopi diversi (descrivere router, reti, route esterne, etc.).
- **LSDB (Link State Database)** : è la struttura organizzata in cui i router immagazzinano tutti gli LSA ricevuti. I routers "inondano" la rete con LSAs finché tutti i nodi della regione OSPF hanno la stessa mappa di rete (cioè lo stesso LSDB sincronizzato).

Il testo semplifica il processo di lavoro di OSPF in tre operazioni principali: collegarsi con gli altri router connessi sullo stesso segmento; scambiare LSA con i vicini; ogni router calcola indipendentemente il percorso migliore (con Dijkstra) verso ogni altra destinazione e inserisce i percorsi migliori nella tabella di routing.

81. A cosa servono le aree in OSPF? Quali regole devono rispettare?

Una rete OSPF, soprattutto se molto estesa, può essere suddivisa in sottodomini chiamati **aree**. Un'area è una raccolta logica di reti OSPF, router e collegamenti che hanno la stessa identificazione di area.

A cosa servono: i router all'interno di un'area devono gestire i dati riguardanti la topologia della **sola area a cui appartiene**; questo fa sì che le dimensioni del database (LSDB) siano ridotte rispetto alla conoscenza dell'intera rete. Le aree limitano l'ambito di distribuzione delle informazioni di route.

Regole che devono rispettare:

1. Deve esistere un'area backbone (area 0), che collega un insieme di aree indipendenti in un singolo dominio.
2. Ogni area non backbone deve essere connessa **direttamente** all'area backbone.
3. L'area della backbone non può essere frammentata in parti non collegate tra loro in condizioni di errore (disconnessione di un router o di un collegamento).

Sezione 5.5 – Congestione

82. Cosa si intende per congestione della rete?

La **congestione della rete** è il fenomeno che si verifica quando i nodi sorgenti immettono nella rete **più pacchetti di quanti essa riesca a consegnare a destinazione**. In condizioni normali, la rete è in grado di gestire il traffico e i pacchetti vengono consegnati con ritardi accettabili. Quando il carico supera la capacità della rete, si instaura un circolo vizioso: i buffer si riempiono, i pacchetti vengono scartati, le ritrasmissioni aumentano il carico, peggiorando ulteriormente la situazione.

83. Quali sono le principali cause della congestione?

La congestione può avere molteplici cause, tra cui:

1. **Diversi flussi attraversano la stessa linea** e i pacchetti che arrivano a un router non riescono a essere processati; si forma quindi una **coda** nel router. Man mano che la coda cresce, i tempi di attesa aumentano e lo spazio di buffer si riduce.

2. **L'esaurimento dei buffer nei router** innesca un meccanismo che porta allo scarto dei pacchetti successivi. Quando i buffer sono pieni, i pacchetti in arrivo vengono scartati (tail drop), causando ritrasmissioni che peggiorano ulteriormente la congestione.
3. **La lentezza della CPU del router** nell'analizzare i pacchetti, consultare la tabella di routing, o per attività di logging degli eventi può portare a rallentamenti nella risposta ed esaurimento della memoria.

84. Quali due parametri consentono di riscontrare l'insorgere di congestione?

I due parametri che consentono di riscontrare l'insorgere di congestione sono:

1. **Il ritardo (latenza)** : in condizioni normali, il ritardo è minimo ed è dovuto solo al ritardo di propagazione e al tempo di processamento nei router. Man mano che il traffico aumenta, anche il ritardo cresce a causa dell'aumento delle code nei router e dei tempi di processamento. In caso di congestione, il ritardo può aumentare drammaticamente fino a diventare inaccettabile.
2. **Il throughput (la quantità di dati consegnati)** : quando la rete lavora in condizione di carico normale, il throughput cresce proporzionalmente al traffico. Se il traffico supera una percentuale piuttosto alta della capacità del canale (intorno all'85-90%), allora il throughput comincia a **diminuire** (le cause sono le stesse elencate sopra: code, scarti, ritrasmissioni).

85. Come si comportano ritardo e throughput al crescere del traffico?

Il testo descrive il comportamento come segue:

- **Ritardo**: è minimo inizialmente (dovuto solo a propagazione e processamento). Man mano che il traffico aumenta, il ritardo cresce gradualmente a causa della formazione di code nei router. In prossimità della congestione, il ritardo cresce più rapidamente, e in condizioni di grave congestione può diventare molto elevato e variabile.
- **Throughput**: cresce in modo proporzionale al traffico fino a un certo punto (tipicamente l'85-90% della capacità del canale). Oltre questa soglia, il throughput comincia a **diminuire** nonostante l'aumento del traffico immesso. Questo accade perché i pacchetti scartati causano ritrasmissioni, che consumano ulteriore banda senza contribuire al throughput utile.

In altre parole, oltre un certo punto, immettere più traffico nella rete porta a consegnare **meno** dati utili.

Sottosezione 5.5.1 – Controllo Proattivo

86. In cosa consiste l'approccio proattivo al controllo della congestione?

L'approccio **proattivo** (o a ciclo aperto) al controllo della congestione consiste in tecniche che cercano di **prevenire la congestione** prima che si verifichi. Il controllo viene fatto o dalla sorgente o dalla destinazione. Non c'è un vero e proprio feedback in tempo reale; invece, si progetta il sistema in modo da evitare che si creino condizioni di congestione. Esempi includono politiche di ritrasmissione ottimizzate, gestione delle finestre, politiche di eliminazione pacchetti preventive, e politiche di accesso.

87. Quali rischi comporta una politica di ritrasmissione aggressiva in termini di congestione?

Una politica di ritrasmissione aggressiva comporta il rischio di **aumentare la congestione** sulla rete. Se il mittente ritrasmette i pacchetti troppo frequentemente o troppo velocemente dopo un timeout (senza attendere un tempo adeguato), può immettere nella rete un numero eccessivo di copie degli stessi pacchetti, peggiorando la congestione già in atto. Per questo motivo, i timer di ritrasmissione devono essere progettati con molta cura, ottimizzando al massimo l'efficienza. Queste tecniche vengono molto usate anche a livello trasporto (TCP).

88. Perché il meccanismo di finestra scorrevole con ripetizione selettiva è preferibile a Go Back N in presenza di congestione?

Il meccanismo di finestra scorrevole con **ripetizione selettiva** (Selective Repeat) invia un numero minore di pacchetti sul canale rispetto al **Go Back N**. In Go Back N, se un pacchetto viene perso, il mittente deve ritrasmettere l'intero blocco di pacchetti a partire da quello perso (anche quelli già ricevuti correttamente). Questo aumenta inutilmente il carico sulla rete in condizioni già congestionate. Con la ripetizione selettiva, il mittente ritrasmette **solo** i pacchetti effettivamente persi, riducendo il traffico superfluo e quindi mitigando la congestione.

89. In che modo le politiche di eliminazione dei pacchetti nei router possono prevenire la congestione?

Le politiche di eliminazione dei pacchetti all'interno dei router sono senz'altro una tecnica che aiuta sia a prevenire che a moderare la congestione. A volte i router sono costretti a eliminare preventivamente i pacchetti per evitare la congestione. Ad esempio, è possibile eliminare alcuni pacchetti di una trasmissione audio che poi verranno ricostruiti per interpolazione dal destinatario. In questo modo, si riduce il carico sulla rete senza compromettere gravemente la qualità percepita dall'utente. Eliminando i pacchetti in modo intelligente (ad esempio, quelli meno importanti o quelli di flussi che possono tollerare perdite), si può prevenire che i buffer si riempiano completamente, evitando il tail drop indiscriminato e le conseguenti ritrasmissioni.

90. Come funzionano le politiche di accesso per reti a circuito virtuale nella prevenzione della congestione?

Le politiche di accesso per reti a circuito virtuale (reti orientate alla connessione) sfruttano i meccanismi studiati per la QoS. In queste reti, i router coinvolti nella comunicazione devono indicare la **disponibilità o meno delle risorse richieste** dal circuito virtuale (il canale di comunicazione) che mittente e destinatario vogliono instaurare. In caso di risorse scarse (possibile congestione), meglio che i router **neghino l'accesso** alla rete. In pratica, prima che la comunicazione inizi, viene eseguita una fase di setup in cui si verifica se ci sono abbastanza risorse (banda, buffer, capacità di processamento) lungo il percorso per supportare il nuovo flusso. Se non ci sono risorse sufficienti, la richiesta di connessione viene rifiutata, prevenendo così la congestione.

Sottosezione 5.5.2 – Controllo Reattivo

91. In cosa consiste l'approccio reattivo al controllo della congestione?

L'approccio **reattivo** (o a ciclo chiuso) al controllo della congestione viene messo in atto **quando la congestione si è già verificata** e bisogna attuare un qualche meccanismo che sia volto a eliminarla o,

quantomeno, ad attenuarla. Invece di prevenire la congestione, si reagisce ad essa implementando azioni correttive come la segnalazione al mittente, la pressione all'indietro, o la segnalazione implicita/esplicita.

92. Cosa sono i choke packets? Come funzionano?

I **choke packets** (pacchetti di strozzamento) sono un meccanismo reattivo di controllo della congestione. Quando un router si accorge della congestione (ad esempio, le sue code superano una certa soglia), per diminuire il carico rimanderà indietro **solo i pacchetti choke** al mittente. Il funzionamento è:

1. Il router rileva congestione sui suoi buffer di uscita.
2. Il router invia un pacchetto di segnalazione (choke packet) direttamente al mittente.
3. Il mittente, ricevuto il choke packet, riduce la sua velocità di trasmissione verso quella destinazione (ad esempio, riducendo la finestra di trasmissione).
4. Dopo un certo tempo, se la congestione persiste, il router può inviare ulteriori choke packet.

Il testo avverte che questa tecnica, se usata in modo non attento, può contribuire a far aumentare la congestione (ad esempio, i choke packet stessi consumano banda di rete).

93. Cosa fa il meccanismo Source Quench in TCP/IP? Perché oggi non viene quasi mai usato?

Nella suite TCP/IP, il meccanismo di segnalazione della congestione può essere fatto usando il protocollo **ICMP** e il meccanismo **Source Quench** (spegni la sorgente). Il Source Quench notifica al mittente che i buffers del router sono sovraccarichi. Quando il messaggio viene ricevuto dal mittente, esso riduce la sua finestra di invio verso la destinazione 'in difficoltà', limitando il traffico in uscita.

Perché oggi non viene quasi mai usato: l'abuso di questo meccanismo potrebbe portare a un **DoS (Denial of Service)**. Un attaccante potrebbe inviare falsi messaggi Source Quench a un mittente legittimo, facendogli ridurre arbitrariamente la velocità di trasmissione o addirittura interrompere la comunicazione. Per questo motivo, nella moderna Internet, Source Quench non viene quasi mai utilizzato.

94. In cosa consiste la tecnica della "pressione all'indietro" (backpressure)? Quale effetto collaterale può avere?

La **pressione all'indietro (backpressure)** è una tecnica in cui la congestione si propaga, da nodo a nodo, nella direzione opposta al flusso di dati. Il funzionamento: capita che la congestione non sia presente alla sorgente ma durante il tragitto. Un nodo congestionato smette di ricevere pacchetti dal nodo che lo precede. Questo nodo a sua volta, non potendo inoltrare i pacchetti, accumula codice e può a sua volta diventare congestionato, propagando la condizione a monte (verso la sorgente).

Effetto collaterale: la pressione all'indietro può **spostare la congestione verso i nodi upstream** (ossia nei nodi predecessori di quello che ha iniziato la backpressure). Invece di risolvere la congestione, la si trasferisce altrove, potenzialmente coinvolgendo più nodi e peggiorando la situazione.

95. Qual è la differenza tra segnalazione implicita e segnalazione esplicita della congestione?

- **Segnalazione implicita:** non c'è comunicazione esplicita tra i nodi congestionati e la sorgente. La fonte **suppone** che ci sia una congestione nella rete in base ad alcuni eventi osservabili:
 - Quando il mittente invia diversi pacchetti e non viene ricevuta alcuna conferma prima della scadenza del timer (timeout).

- La ricezione di duplicati di conferme (acknowledgement duplicati) può anche essere indice di congestione nella rete.
- **Segnalazione esplicita:** se un nodo riscontra una congestione, invia **esplicitamente** un pacchetto alla sorgente o alla destinazione per informare della congestione. La differenza tra choke packet e segnalazione esplicita è che l'avviso di congestione fa parte dei pacchetti che trasportano i dati (ad esempio, attraverso l'impostazione di un flag nell'header) anziché crearne uno ad hoc come nel caso della tecnica choke packet.

96. Che differenza c'è tra segnalazione diretta (forward) e segnalazione all'indietro (backward)?

La segnalazione esplicita può avvenire sia in avanti che all'indietro:

- **Segnalazione diretta (forward)** : l'avviso di congestione viene inviato nella direzione del flusso principale (quello in cui si riscontra il problema). La destinazione viene avvisata della congestione. Il destinatario in questo caso adotta politiche volte a prevenire ulteriori congestioni (ad esempio, potrebbe rallentare l'invio di acknowledgment o richiedere al mittente di rallentare).
- **Segnalazione all'indietro (backward)** : un avviso viene inviato nella direzione opposta al flusso dei dati. La fonte (mittente) è avvisata della congestione e deve rallentare l'invio. È il caso dei choke packet e del Source Quench.

Sottosezione 5.5.3 – Leaky Bucket / Token Bucket

97. A cosa servono gli algoritmi Leaky Bucket e Token Bucket?

I due metodi per 'modellare' il traffico (traffic shaping) e ridurre la congestione sono l'algoritmo del **leaky bucket** (secchio bucato) e quello del **token bucket** (secchio dei gettoni). Servono a controllare sia la quantità totale di traffico che la velocità con cui esso viene inviato alla rete, riuscendo anche a rilevare sia l'aumento graduale che i picchi repentini di pacchetti nel buffer. In pratica, consentono di "plasmare" un flusso di traffico da una forma burst (a raffica) a una forma più regolare, riducendo la probabilità di congestione a valle.

98. Spiega il funzionamento dell'algoritmo Leaky Bucket usando l'analogia del secchio bucato.

L'algoritmo del leaky bucket funziona in maniera simile al modo in cui un secchio bucato trattiene l'acqua:

- L'**acqua** rappresenta i pacchetti (che possono arrivare fino a una capacità massima) che entra dall'alto nel secchio (input).
- Il **buco sul fondo** rappresenta l'uscita a velocità costante (output).
- Se il flusso dati in entrata è troppo alto e i pacchetti riempissero il secchio, quelli in eccesso vengono considerati "non conformi" e quindi **scartati**.
- I pacchetti ritornano ad essere ammessi nel secchio non appena esso ritorna ad avere spazio svuotandosi.
- L'algoritmo del leaky bucket è ideale per livellare il traffico di tipo burst ('sprazzi' di traffico al massimo della banda) in traffico a velocità fissa.

Si può implementare l'algoritmo con un buffer gestito in modalità FIFO: la coda trattiene i pacchetti e non consente loro di passare se superano la larghezza di banda settata sulla rete in uscita.

99. Cosa significa "tail drop" nel contesto del Leaky Bucket?

Il **tail drop** (coda a scarto in coda) è un fenomeno che si verifica nell'algoritmo Leaky Bucket quando il traffico continua ad arrivare e il buffer è pieno: l'algoritmo scarterà i pacchetti appena ricevuti (quelli in coda, cioè gli ultimi ad arrivare). Il testo specifica che questo fenomeno avviene **indipendentemente dall'importanza del pacchetto o dal flusso a cui esso appartiene**. In altre parole, tutti i pacchetti sono trattati allo stesso modo, e quando il buffer è pieno, i nuovi pacchetti vengono semplicemente scartati senza alcuna distinzione. Questo può essere problematico perché non differenzia flussi importanti da flussi meno importanti.

100. Quale limite ha il Leaky Bucket che il Token Bucket supera?

Il limite dell'algoritmo del leaky bucket è che modella il traffico a raffica in traffico a velocità fissa calcolando la media della velocità dati **ma non tiene conto del tempo di inattività di un host inattivo**. Se un host rimane inattivo per un certo periodo, non accumula "credito" per trasmettere successivamente una raffica più grande. Il leaky bucket impone un rate costante indipendentemente dalla storia passata.

L'algoritmo del **token bucket** consente invece all'host, durante l'inattività, di **accumulare credito per il futuro** sotto forma di token, superando così il limite del leaky bucket. Nel token bucket si possono inviare pacchetti anche alla massima velocità se ci sono abbastanza token a disposizione.

101. Spiega il funzionamento dell'algoritmo Token Bucket.

L'algoritmo del token bucket funziona come segue:

- I token vengono aggiunti al bucket a un **rate costante** (ad esempio, r token al secondo).
- Il bucket ha una **capacità massima** (B , il numero massimo di token che può contenere).
- Ogni volta che si spedisce un pacchetto, vengono **consumati** (toliti) un numero di token pari alla dimensione del pacchetto (o tipicamente 1 token per pacchetto, a seconda della definizione).
- Se ci sono abbastanza token, il pacchetto viene trasmesso immediatamente.
- Se il bucket è vuoto (non ci sono token sufficienti), il pacchetto deve attendere che vengano generati nuovi token (o viene scartato, a seconda della politica).
- Durante i periodi di inattività, il bucket si riempie fino alla capacità massima, accumulando token che possono essere utilizzati per gestire successivi burst di traffico.

Questo meccanismo permette di avere un rate medio di r token al secondo, ma consente anche burst di dimensione massima B (perché si possono consumare tutti i token accumulati in una volta).

102. Quali sono i principali svantaggi del Token Bucket rispetto al Leaky Bucket?

Secondo il testo, il token bucket presenta alcuni svantaggi:

1. **Se il bucket dei token è vuoto**, i pacchetti dovranno attendere nuovi token, con conseguente **aumento della latenza** e potenziale perdita di pacchetti (se l'attesa è troppo lunga o se i buffer di attesa si riempiono).

2. **È meno flessibile del leaky bucket** quando si tratta di modellare il traffico di rete a causa della velocità fissa di generazione dei token che non può essere facilmente modificata per soddisfare i mutevoli requisiti della rete. Il leaky bucket, essendo più semplice, può essere più facile da adattare in alcune situazioni.

In generale, la scelta tra leaky bucket e token bucket dipende dalle caratteristiche del traffico e dagli obiettivi di qualità del servizio.

Sottosezione 5.5.4 – Load Shedding

103. Cosa si intende per load shedding (riduzione del carico)?

Il **load shedding** (riduzione del carico) è un'altra tecnica utilizzata per il controllo della congestione e interviene sui **buffer del router**. Sapendo che i buffer sono utilizzati per conservare i pacchetti e poi instradarli verso la loro destinazione, la riduzione del carico è definita come un approccio che consiste nello **scartare i pacchetti quando il buffer è pieno** secondo una politica implementata a livello di collegamento. Invece di scartare semplicemente l'ultimo pacchetto arrivato (tail drop), si applicano politiche più intelligenti per decidere quali pacchetti scartare.

104. Qual è la differenza tra la "wine policy" e la "milk policy" nella scelta dei pacchetti da scartare? Per quale tipo di applicazione è adatta ciascuna?

- **Wine policy** (politica del vino): "più vecchio è, meglio è". Per il trasferimento di file, un vecchio pacchetto vale più di uno nuovo. Questo perché eliminare un vecchio pacchetto potrebbe forzare la ritrasmissione di altri pacchetti (se si tratta di un file sequenziale). Inoltre il file risulterebbe inservibile in caso di 'buchi' nelle informazioni. Per questo tipo di applicazioni, quando si deve scartare, è meglio scartare i pacchetti più recenti.
- **Milk policy** (politica del latte): "più fresco è, meglio è". Per il traffico multimediale (es. comunicazione telefonica digitale, VoIP), un nuovo pacchetto è più importante di uno vecchio. In applicazioni in tempo reale, un pacchetto vecchio (in ritardo) è inutile perché l'evento che rappresenta è già passato; è meglio scartare i pacchetti in ritardo e consegnare solo quelli nuovi. Per questo tipo di applicazioni, quando si deve scartare, è meglio scartare i pacchetti più vecchi.

105. In quale campo dell'header IP il mittente può contrassegnare la priorità del pacchetto?

Il mittente può contrassegnare la priorità del pacchetto usando il campo **ToS (Type of Service)** all'interno dell'header IP. A seconda della priorità settata, il pacchetto può essere selezionato o scartato. La priorità può essere assegnata in base al costo, all'algoritmo o ai servizi a cui appartiene.

106. Cosa significa l'acronimo RED (Random Early Detection)? Di che tipo di approccio si tratta (proattivo/reattivo)?

RED significa **Random Early Detection** (rilevazione precoce casuale). È un altro metodo molto usato (a livello rete) per decidere quale pacchetto scartare in caso di congestione. Il testo specifica che RED **ha un comportamento proattivo** in quanto cerca di **prevenire la congestione**, invece di reagire quando è già troppo tardi. L'idea è di iniziare a scartare pacchetti (o a marcarli) **prima** che i buffer si riempiano completamente, per segnalare in modo graduale la congestione e dare ai mittenti il tempo di rallentare.

107. Quali sono i punti essenziali dell'algoritmo RED?

I punti essenziali dell'algoritmo RED sono:

1. **Per rilevare le condizioni di congestione si utilizza la dimensione dei buffer di uscita** (più precisamente, una media pesata della lunghezza della coda).
2. **Per prevenire la congestione si marcano i pacchetti in uscita.** Rispetto ad altri metodi, RED introduce alcune migliorie che ne aumentano le prestazioni.
3. **Non si considera la dimensione istantanea della coda**, ma una forma di **media pesata** (EWMA). Questo consente di ignorare picchi episodici, e trattare solo picchi di traffico persistenti.
4. In base all'hardware e ai flussi di dati, vengono scelte **due soglie**, una **minima** e una **massima**.
5. Quando la lunghezza media della coda supera la soglia minima, **alcuni pacchetti vengono marcati** (scelti a caso, in numero crescente con la lunghezza media).
6. Oltre la soglia massima, **tutti i pacchetti sono marcati**.

108. Perché RED utilizza una media pesata della lunghezza della coda invece del valore istantaneo?

RED utilizza una media pesata (EWMA - Exponentially Weighted Moving Average) della lunghezza della coda invece del valore istantaneo per **ignorare picchi episodici** e trattare solo picchi di traffico persistenti. La lunghezza istantanea della coda può variare molto rapidamente a causa di fluttuazioni statistiche del traffico. Se si reagisse a ogni picco istantaneo, si potrebbero marcare o scartare pacchetti inutilmente, causando instabilità. Usando una media pesata, l'algoritmo "filtra" le variazioni a breve termine e reagisce solo a cambiamenti sostenuti nel livello di congestione, che sono quelli che realmente richiedono un intervento.

109. Cosa succede quando la lunghezza media della coda supera la soglia minima in RED? E quando supera la soglia massima?

- **Quando la lunghezza media della coda supera la soglia minima:** alcuni pacchetti vengono marcati. I pacchetti marcati vengono scelti a caso, e in un numero che **cresce con la lunghezza media della coda**. Più la coda è lunga, maggiore è la probabilità che un pacchetto venga marcato. Questo crea un feedback graduale: i mittenti iniziano a ricevere segnali di congestione (attraverso la marcatura) prima che la congestione diventi grave.
- **Quando la lunghezza media della coda supera la soglia massima:** **tutti i pacchetti sono marcati** (o scartati, a seconda della configurazione). Questo rappresenta una situazione di congestione grave in cui si ritiene che tutti i pacchetti dovrebbero essere informati della congestione (o scartati per proteggere la rete).

110. In quali due modi può consistere la "marcatura" di un pacchetto in RED?

La marcatura del pacchetto è un'operazione in un certo senso simbolica. A seconda della tecnologia del router, tale marcatura può consistere:

1. **Nella rimozione del pacchetto** (drop): il pacchetto viene scartato. Questa è l'azione più drastica, che segnala la congestione al mittente attraverso la mancata ricezione di un acknowledgement (a livello trasporto).
2. **Nell'impostazione di un bit o campo nel pacchetto** (ECN - Explicit Congestion Notification): il pacchetto non viene scartato, ma viene "segnato" (ad esempio, impostando due bit specifici nell'header IP). Il destinatario, ricevendo il pacchetto marcato, può informare il mittente della congestione attraverso un meccanismo di feedback (ad esempio, TCP ECN). Questo approccio evita la perdita di pacchetti.

In ogni caso, si intende che la marcatura avrà un effetto di feedback sul mittente del pacchetto.

Domande trasversali e di confronto

111. Confronta IPv4 e IPv6 in termini di: lunghezza dell'indirizzo, dimensione dell'header, presenza del checksum, supporto al broadcast, sicurezza nativa.

Caratteristica	IPv4	IPv6
Lunghezza indirizzo	32 bit	128 bit
Dimensione header	20-24 byte (variabile)	40 byte (fisso)
Checksum header	Presente (ricalcolato a ogni hop)	Rimosso (per velocizzare)
Supporto broadcast	Presente	Eliminato (sostituito da multicast)
Sicurezza nativa	Affidata a protocolli esterni (IPsec opzionale, poi reso comune)	AH e ESP integrati come extension header; IPsec obbligatorio nelle implementazioni

112. Confronta gli algoritmi Distance Vector e Link State in termini di: conoscenza della topologia, velocità di convergenza, rischio di loop, scalabilità, consumo di risorse.

Caratteristica	Distance Vector	Link State
Conoscenza topologia	Solo distanze verso destinazioni (non la mappa completa)	Mappa completa della rete (topologia)
Velocità di convergenza	Lenta (soggetta a counting to infinity)	Veloce (flooding + Dijkstra)
Rischio di loop	Sì (counting to infinity)	No (ogni nodo ha visione coerente)
Scalabilità	Buona (adatta a reti grandi)	Limitata (non scalabile in reti molto vaste)

Caratteristica	Distance Vector	Link State
Consumo risorse	Basso (solo vettori di distanze)	Alto (mappa completa + calcoli Dijkstra)

113. Spiega la differenza tra controllo proattivo e reattivo della congestione. Fornisci almeno due esempi per ciascuna categoria.

- **Controllo proattivo (a ciclo aperto)** : cerca di **prevenire** la congestione prima che si verifichi. Non si basa su feedback in tempo reale, ma sulla progettazione di politiche che evitano la congestione.
 - Esempi: politiche di ritrasmissione ottimizzate, gestione della finestra scorrevole, politiche di eliminazione pacchetti preventive, politiche di accesso (negoiazione risorse).
- **Controllo reattivo (a ciclo chiuso)** : interviene **dopo che la congestione si è verificata** per eliminarla o mitigarla.
 - Esempi: choke packet, pressione all'indietro (backpressure), segnalazione implicita (timeout/duplicati), segnalazione esplicita (forward/backward).

114. Confronta l'algoritmo Leaky Bucket e Token Bucket in termini di gestione dei burst e del tempo di inattività.

Caratteristica	Leaky Bucket	Token Bucket
Gestione burst	Livella i burst in traffico a velocità costante (non permette burst in uscita superiori al rate)	Permette burst in uscita fino alla capacità massima del bucket (se ci sono token accumulati)
Tempo di inattività	Non tiene conto dell'inattività: il rate in uscita è sempre costante	Accumula token durante l'inattività, permettendo burst successivi
Forma del traffico in uscita	Traffico regolare (a rate costante)	Traffico a rate medio r ma con burst consentiti
Perdita di pacchetti	Quando il bucket è pieno, i pacchetti vengono scartati (tail drop)	Quando il bucket token è vuoto, i pacchetti attendono o vengono scartati

115. Quali sono le differenze principali tra indirizzi unicast, multicast e anycast in IPv6?

- **Unicast**: identifica una singola interfaccia (singolo dispositivo). Un pacchetto inviato a un indirizzo unicast viene consegnato a quella specifica interfaccia. È il tipo di indirizzamento più comune.
- **Multicast**: identifica un gruppo di interfacce (un insieme di dispositivi). Un pacchetto inviato a un indirizzo multicast viene consegnato a **tutte** le interfacce del gruppo. Il prefisso è ff00::/8.
- **Anycast**: identifica un insieme di interfacce (come multicast), ma un pacchetto inviato a un indirizzo anycast viene consegnato a **una sola** di esse (tipicamente la più vicina in termini di routing). È utile per servizi replicati (es. root DNS server).

116. Perché il NAT è molto comune in IPv4 ma non necessario in IPv6?

Il NAT (Network Address Translation) è molto comune in IPv4 a causa della **penuria di indirizzi** IPv4 (solo 2^{32} indirizzi, molti dei quali non utilizzabili o riservati). Con IPv4, non è possibile assegnare un indirizzo pubblico univoco a ogni dispositivo connesso a Internet. Il NAT permette di condividere un singolo indirizzo pubblico tra molti dispositivi con indirizzi privati.

In IPv6, lo spazio di indirizzi è immenso (2^{128} indirizzi), sufficiente per assegnare un indirizzo pubblico univoco a ogni dispositivo (e anche a molti di più). In IPv6, l'idea è che ciascun dispositivo abbia **almeno un indirizzo pubblico visibile su Internet** (global unicast). Per questo motivo, il NAT non è necessario in IPv6. Si ripristina così il principio di **connettività end-to-end** che in IPv4 era stato compromesso proprio dal NAT.